

CYPRUS INTERNATIONAL UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

**MULTIMODAL HUMAN COMPUTER
INTERACTION USING GESTURE AND VOICE
RECOGNITION**

by

Ayomiposi ADEBAYO

Fiyinfooluwa Emmanuel AKINBO

Hafeez AHMED

Idiat Abolaji ADETUNJI

Wael Sabe ALLIL

April, 2025

Nicosia, NORTH CYPRUS

CYPRUS INTERNATIONAL UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

**MULTIMODAL HUMAN COMPUTER
INTERACTION USING GESTURE AND VOICE
RECOGNITION**

by

Ayomiposi Pelumi ADEBAYO

Fiyinfooluwa Emmanuel AKINBO

Hafeez AHMED

Idiat Abolaji ADETUNJI

Wael Sabe ALLIL

April, 2025

Nicosia, NORTH CYPRUS

**MULTIMODAL HUMAN COMPUTER INTERACTION
USING GESTURE AND VOICE RECOGNITION**

By

**AYOMIPOSİ ADEBAYO
FIYINFOLUWA EMMANUEL AKINBO
HAFEEZ AHMED
IDIAT ABOLAJI ADETUNJI
WAEŁ SABLE ALLİL**

DATE OF APPROVAL:

APPROVED BY:

(Asst./Assoc.) Prof.Dr.
Project Supervisor

Melike Şah Direcoglu

ABSTRACT

Traditional human–computer interaction relies heavily on physical input devices such as keyboards, mice, and touchscreens, which can limit accessibility, efficiency, and natural user interaction. This project presents the design and implementation of a multimodal human–computer interaction (HCI) system that enables users to control a computer using a combination of hand gestures and voice commands. By integrating visual and auditory input modalities, the system aims to provide a more intuitive, flexible, and accessible interaction experience.

The gesture recognition component utilizes real-time computer vision techniques to detect and interpret hand movements captured through a standard webcam. A custom-trained deep learning model is used to recognize static and dynamic gestures, which are mapped to system-level actions such as cursor movement, clicking, scrolling, and application navigation. To ensure smooth and precise control, adaptive filtering and motion smoothing techniques are applied to reduce jitter and improve tracking accuracy. In parallel, the voice recognition component employs an offline speech recognition pipeline with a wake-word mechanism to continuously listen for user commands. Natural language understanding is used to interpret spoken instructions and convert them into executable system actions, including application control, media playback, system settings adjustment, and web searches. A unified dispatcher architecture enables seamless integration of both modalities while supporting cross-platform compatibility for macOS and Windows environments.

The final system demonstrates that combining gesture and voice input significantly enhances usability compared to single-modality interfaces. The results highlight the potential of multimodal HCI systems in improving accessibility, hands-free interaction, and natural user experiences, particularly in assistive technology, smart environments, and future computing interfaces.

ÖZET

Geleneksel insan–bilgisayar etkileşimi, klavye, fare ve dokunmatik ekran gibi fiziksel giriş aygıtlarına dayanmaktadır. Bu yöntemler, özellikle erişilebilirlik, kullanım kolaylığı ve doğal etkileşim açısından çeşitli sınırlamalar oluşturmaktadır. Bu çalışma, sesli komutlar ve el hareketleri kullanarak bilgisayar kontrolünü mümkün kılan çok kipli (multimodal) bir insan–bilgisayar etkileşim sistemi geliştirmeyi amaçlamaktadır. Önerilen sistem, görsel ve işitsel girdileri birleştirerek daha sezgisel, esnek ve doğal bir kullanıcı deneyimi sunmaktadır.

Geliştirilen sistemin el hareketi tanıma bileşeni, standart bir web kamerasından elde edilen görüntüler üzerinde gerçek zamanlı bilgisayarlı görü teknikleri kullanılarak gerçekleştirilmiştir. Statik ve dinamik el hareketleri, özel olarak eğitilmiş derin öğrenme modelleri aracılığıyla sınıflandırılmış ve imleç hareketi, tıklama, kaydırma ve uygulama geçişleri gibi sistem düzeyindeki eylemlere dönüştürülmüştür. İmleç kontrolünde kararlılığı artırmak ve titreşimi azaltmak amacıyla adaptif filtreleme ve hareket yumuşatma teknikleri uygulanmıştır. Sesli etkileşim bileşeni ise çevrimdışı çalışan bir konuşma tanıma altyapısı ve uyandırma kelimesi (wake-word) mekanizması kullanılarak tasarlanmıştır. Doğal dil işleme yöntemleriyle kullanıcı komutları yorumlanmış ve uygulama kontrolü, medya oynatma, sistem ayarları ve internet aramaları gibi işlemler gerçekleştirilmiştir. Çok kipli mimari, macOS ve Windows işletim sistemlerini destekleyecek şekilde platformdan bağımsız olarak yapılandırılmıştır.

Elde edilen sonuçlar, ses ve jest girdilerinin birlikte kullanılmasının tek kipli sistemlere kıyasla daha etkili ve kullanıcı dostu bir etkileşim sağladığını göstermektedir. Bu çalışma, özellikle yardımcı teknolojiler, akıllı ortamlar ve geleceğin etkileşim sistemleri için çok kipli insan–bilgisayar etkileşiminin potansiyelini ortaya koymaktadır.

TABLE OF CONTENT

ABSTRACT.....	3
ÖZET.....	4
CHAPTER ONE.....	9
INTRODUCTION.....	9
1.1 PROBLEM STATEMENT.....	9
1.2 MOTIVATION.....	10
1.3 OBJECTIVES.....	11
1.4 SCOPE.....	12
1.6 ORGANIZATION OF THE REPORT.....	13
CHAPTER TWO.....	15
LITERATURE SURVEY.....	15
2.1 VOICE-BASED HUMAN-COMPUTER INTERACTION.....	15
2.2 GESTURE-BASED HUMAN-COMPUTER INTERACTION.....	15
2.3 MULTIMODAL INTERACTION SYSTEM.....	16
2.4 RESEARCH GAP AND MOTIVATION.....	16
CHAPTER THREE.....	17
REALISTIC CONSTRAINTS.....	17
3.1 TECHNICAL CONSTRAINTS.....	17
3.2 ECONOMIC CONSTRAINTS.....	18
3.3 ENVIRONMENTAL AND SUSTAINABILITY CONSTRAINTS.....	18
3.4 ETHICAL AND PRIVACY CONSTRAINTS.....	19
3.5 HEALTH AND SAFETY CONSTRAINTS.....	20
3.6 LEGAL AND REGULATORY CONSTRAINTS.....	20
CHAPTER FOUR.....	21
ARCHITECTURE.....	21
4.1 ELECTRON LAYER.....	21
4.2 FRONT-END INTERFACE.....	22
4.3 BACKEND ENGINE.....	22
4.3.1 VOICE MODULE.....	22
4.3.2 GESTURE MODULE.....	24
4.3.3 DISPATCHER MODULE.....	25
4.4 ORCHESTRATION AND CONTEXT.....	25
4.5 OVERALL INTERACTION FLOW.....	25
CHAPTER FIVE.....	27
SYSTEM TOOLS, TECHNOLOGIES, AND DATA COLLECTION.....	27

5.1 OVERVIEW.....	27
5.2 SOFTWARE TOOLS AND DEVELOPMENT.....	27
5.3 LIBRARIES AND FRAMEWORK USED.....	28
5.3.1 VOICE RECOGNITION AND PROCESSING LIBRARIES.....	28
5.3.2 Gesture Recognition Libraries.....	29
5.4 GESTURE DATA COLLECTION METHODOLOGY.....	30
5.4.1 DATE CAPTURE PROCESS.....	30
5.4.2 LABELLING AND STORAGE.....	30
5.4.3 DATASET VALIDATION.....	31
5.5 TOOLS USED FOR THE FRONT-END APPLICATION.....	31
CHAPTER SIX.....	32
VOICE SYSTEM IMPLEMENTATION.....	32
6.1 OVERVIEW OF THE VOICE INTERACTION PIPELINE.....	32
6.2 WAKE WORD DETECTION AND RUNTIME CONTROL.....	33
6.2.1 CONTINUOUS AUDIO STREAMING.....	33
6.2.2 WAKE WORD DETECTION USING PICOVOICE PORCUPINE.....	34
6.2.3 USER FEEDBACK AND STATE TRANSITION.....	34
6.3 SPEECH RECORDING AND VOICE ACTIVITY DETECTION.....	34
6.3.1 CONTROLLED COMMAND RECORDING.....	34
6.3.2 ENERGY-BASED SILENCE DETECTION.....	35
6.3.3 IN-MEMORY AUDIO ENCODING.....	35
6.4 SPEECH-TO-TEXT TRANSCRIPTION.....	36
6.4.1 API KEY MANAGEMENT AND SECURITY.....	36
6.4.2 AUDIO UPLOAD PROCESS.....	36
6.4.3 TRANSCRIPTION JOB LIFECYCLE.....	37
6.5 NATURAL LANGUAGE UNDERSTANDING.....	37
6.5.1 USE OF A LARGE LANGUAGE MODEL.....	38
6.5.2 STRICT OUTPUT ENFORCEMENT AND VALIDATION.....	38
6.5.3 MULTI-INTENT COMMAND SUPPORT.....	38
6.6 ACTION DISPATCHING AND EXECUTION.....	39
6.7 SYSTEM INTEGRATION AND CONTROL FLOW.....	39
6.8 CHAPTER SUMMARY.....	40
CHAPTER SEVEN.....	41
GESTURE RECOGNITION SYSTEM IMPLEMENTATION.....	41
7.1 OVERVIEW OF GESTURE IMPLEMENTATION.....	41
7.2 HAND TRACKING AND LANDMARK EXTRACTION.....	42
7.2.1 MEDIA PIPE-BASED HAND DETECTION.....	42

7.3 GESTURE DATA RECORDING.....	42
7.3.1 CONTROLLED DATA CAPTURE.....	43
7.3.2 DATASET ORGANIZATION.....	43
7.4 GESTURE DEFINITION AND CONFIGURATION.....	43
7.5 FEATURE EXTRACTION AND GESTURE CLASSIFICATION.....	44
7.5.1 STATIC GESTURE CLASSIFICATION.....	44
7.5.2 CUSTOM GESTURE CLASSIFICATION.....	45
7.6 REAL-TIME GESTURE RECOGNITION.....	46
7.6.1 FRAME PROCESSING LOOP.....	46
7.6.2 STABILITY AND TEMPORAL FILTERING.....	46
7.7 MOTION-BASED GESTURE AND CURSOR CONTROL.....	47
7.7.1 MOTION GESTURE CLASSIFICATION.....	47
7.7.2 CURSOR MOVEMENT AND CLICK DETECTION.....	47
7.8 GESTURE-CONTROLLED ACTIONS.....	48
7.9 CHAPTER SUMMARY.....	48
CHAPTER EIGHT.....	49
APPLICATION INTERFACE DESIGN AND INTEGRATION.....	49
8.1 OVERVIEW OF THE APPLICATION INTERFACE.....	49
8.2 FRONT-END TECHNOLOGY STACK.....	49
8.3 APPLICATION ARCHITECTURE AND BACKEND INTEGRATION.....	50
8.4 MAIN DASHBOARD DESIGN.....	51
8.4.1 DASHBOARD PURPOSE.....	51
8.4.2 MODE CONTROL AND EXECUTION FLOW.....	51
8.4.3 GESTURE DETECTION PANEL.....	52
8.4.4 VOICE RECOGNITION PANEL.....	53
8.5 LIVE ACTIVITY LOG AND HISTORY VIEW.....	53
8.5.1 LIVE ACTIVITY LOG.....	53
8.5.2 HISTORY PAGE.....	54
8.6 SETTINGS PAGE AND SYSTEM CONFIRMATION.....	55
8.7 TUTORIAL AND USER GUIDANCE INTERFACE.....	55
8.8 USER INTERACTION FLOW SUMMARY.....	57
8.9 CHAPTER SUMMARY.....	57
CHAPTER NINE.....	58
SYSTEM INTEGRATION, TESTING, AND PERFORMANCE	
EVALUATION.....	58
9.1 FRONT-END AND BACK-END INTEGRATION.....	58
9.2 TESTING METHODOLOGY.....	59

9.3 DEBUGGING AND SYSTEM REFINEMENT.....	59
9.4 PERFORMANCE EVALUATION.....	60
9.5 ACCURACY IMPROVEMENTS.....	60
CHAPTER TEN.....	61
CHALLENGES, DESIGN EVOLUTION, AND SYSTEM LIMITS.....	61
10.1 DEVELOPMENT CHALLENGES.....	61
10.2 EVOLUTION OF DESIGN DECISION.....	61
10.3 TRADE-OFFS AND DESIGN CONSTRAINTS.....	62
10.4 SYSTEM LIMITATIONS.....	63
10.5 FUTURE WORK.....	63
10.6 SUMMARY.....	64
CONCLUSION.....	65
REFERENCES.....	67
APPENDIX A.....	69
SAMPLE FIGURES.....	69

TABLE OF FIGURES

Fig1. voice module architecture.....	24
Fig2. gesture module architecture.....	25
Fig3. whole system architecture.....	27
Fig4. voice_main_wake.py.....	34
Fig5. assemblyai_client.py.....	37
Fig6. nlu.py.....	38
Fig7. dispatcher.....	40
Fig8. file gesture_record.py.....	43
Fig9. custom_gestures.json.....	44
Fig10. gesture_classifier.py.....	45
Fig11. custom_gesture_classifier.py.....	46
Fig12. Motion_gesture_classifier.py.....	48
Fig13. Dashboard design.....	52
Fig14. Log and history UI.....	55
Fig15. Setting interface.....	56
Fig16. Tutorial page.....	57
Fig17. Move cursor gesture.....	70
Fig18. Left click gesture.....	71
Fig18. Right click gesture.....	72
Fig19. Screenshot gesture.....	73
Fig20. Continuous scrolling down gesture.....	74
Fig21. Continuous scrolling up gesture.....	75
Fig22. Swipe left motion gesture.....	76
Fig23. Swipe Right motion gesture.....	77

CHAPTER ONE

INTRODUCTION

1.1 PROBLEM STATEMENT

Human-computer interaction has long depended on traditional input devices such as keyboards, mice, and touchscreens. While effective, these interaction methods are not always natural, accessible, or efficient in modern computing environments. Situations that require hands-free operation, faster interaction, or improved accessibility reveal clear limitations in conventional input systems.

With recent advances in artificial intelligence, speech recognition, and computer vision, alternative interaction methods such as voice commands and hand gestures have become increasingly viable. However, most existing systems rely on a single interaction modality, which introduces reliability issues. Voice-only systems struggle in noisy environments, while gesture-only systems can suffer from misclassification, occlusion, and fatigue. Additionally, many solutions are application-specific, cloud-dependent, or lack real-time responsiveness.

This limitation highlights the need for a lightweight, real-time multimodal human-computer interaction system that integrates both voice and gesture recognition. By combining these modalities, the system can improve accuracy, robustness, and usability while providing a more natural and intuitive way for users to interact with their computers. This project aims to address this gap by designing and implementing a unified multimodal interaction framework for desktop environments [1].

1.2 MOTIVATION

We live in a time where computers are everywhere, yet interacting with them still often feels rigid and unnatural. Keyboards and mice work, sure, but they don't always fit every situation. Think hands occupied, accessibility needs, or environments where quick, intuitive interaction matters more than precision clicks. In those moments, traditional input methods start to feel limiting and disconnected from how humans naturally communicate.

That gap is what sparked this project. The goal was not just to build another voice assistant or gesture controller, but to create a system that understands context and adapts accordingly. Voice commands can fail in noisy spaces or when speech is unclear, while gesture recognition alone can be unreliable, physically tiring, or prone to unintended movements. By combining both interaction modes, the system can compensate for the weaknesses of each modality and offer a more dependable and flexible user experience.

Beyond functionality, this project was driven by a desire to make computer interaction feel more natural and inclusive. It provided an opportunity to explore how people instinctively use speech and hand movements together in everyday communication, and how those patterns can be translated into real-time system control. By fusing voice recognition with hand gesture input, the system aims to reduce friction, improve accessibility, and move closer to truly human-centered computing.

1.3 OBJECTIVES

The main objective of this project is to design and implement a lightweight, accurate, and fluid multimodal human–computer interaction system that enables users to control a computing device seamlessly using both voice commands and hand gestures. The system is intended to operate in real time, adapt to different usage contexts, and provide an intuitive alternative to traditional input methods such as keyboards and mice [2].

To achieve this overall goal, the project is structured around the following specific objectives:

- To develop a real-time gesture recognition system capable of detecting and classifying hand movements and static poses using computer vision techniques.
- To implement a voice recognition and natural language understanding pipeline that accurately interprets spoken commands under varying conditions.
- To fuse gesture-based and voice-based inputs into a unified control framework, allowing each modality to complement the other and reduce interaction errors.
- To ensure the system remains lightweight and efficient, enabling smooth performance on standard consumer hardware without requiring specialized equipment.
- To map recognized gestures and voice commands to practical system-level actions such as cursor control, application navigation, media control, and system operations.
- To evaluate the system’s performance in terms of accuracy, responsiveness, reliability, and overall usability during real-time interaction.

1.4 SCOPE

This project focuses on the design and implementation of a multimodal human–computer interaction system that combines voice recognition and hand gesture recognition to control a computing device. The system is developed and evaluated within a software-based, real-time environment running on standard consumer hardware, without the need for specialized sensors or external devices.

Key aspects within the scope of this project include:

- Real-time detection and classification of hand gestures using computer vision techniques.
- Voice command recognition and natural language processing for system control.
- Fusion of voice and gesture inputs to improve interaction accuracy and robustness.
- Mapping recognized commands to practical operating system and application-level actions.
- Evaluation of system performance in terms of accuracy, responsiveness, and usability.

1.6 ORGANIZATION OF THE REPORT

This report is organized to provide a clear, logical, and comprehensive overview of the design, implementation, integration, and evaluation of the proposed multimodal human–computer interaction system based on voice and hand gesture recognition.

Chapter 1 introduces the project by presenting the problem statement, motivation, objectives, and scope. This chapter establishes the context for the research and highlights the need for more natural and intuitive interaction methods beyond traditional input devices.

Chapter 2 presents a literature survey related to multimodal human–computer interaction, voice-based interfaces, gesture recognition systems, and supporting technologies. This chapter reviews existing approaches, identifies gaps in current solutions, and positions the proposed system within the broader research landscape.

Chapter 3 describes the implementation of the voice recognition component. It details the wake-word detection mechanism, audio capture process, speech-to-text transcription, natural language understanding using intent recognition, and the execution of system-level commands.

Chapter 4 focuses on the gesture recognition subsystem. This chapter explains the computer vision pipeline, hand landmark tracking, static and motion-based gesture classification, cursor control, and the mapping of gestures to system actions in real time.

Chapter 5 discusses the design and development of the application interface. It covers the front-end architecture, user interaction flow, dashboard functionality, tutorial and settings pages, and real-time visualization of system status and activity logs.

Chapter 6 examines the integration between the front end and back end of the system. It also presents comprehensive testing and debugging procedures,

performance evaluation, and accuracy improvements applied across both voice and gesture modalities.

Chapter 7 outlines the realistic constraints considered during the project, including technical, economic, environmental, ethical, health, and legal factors that influenced system design and implementation choices.

Chapter 8 presents the overall system evaluation and discussion of results, summarizing performance outcomes, usability observations, and system behavior under real-world conditions.

Chapter 9 discusses the challenges encountered during development, key design decisions that evolved over time, and limitations of the current system. This chapter also identifies potential directions for future work and enhancements.

CHAPTER TWO

LITERATURE SURVEY

2.1 VOICE-BASED HUMAN-COMPUTER INTERACTION

Voice-based interaction is one of the most widely studied natural interaction modalities in HCI. Early systems relied on keyword spotting and fixed command grammars, which required users to memorize specific phrases. With advances in automatic speech recognition (ASR) and natural language processing (NLP), modern voice systems can now process free-form speech and infer user intent. Commercial voice assistants demonstrate the effectiveness of combining speech-to-text with intent recognition; however, many remain closed-source and unsuitable for experimental research. Wake word detection has become a standard technique for enabling continuous listening while preserving efficiency, making voice interaction practical on personal devices.

2.2 GESTURE-BASED HUMAN-COMPUTER INTERACTION

Gesture-based interaction allows users to communicate through physical movements, commonly captured using camera-based systems. Recent progress in computer vision has enabled real-time hand tracking using standard webcams, removing the need for specialized hardware. Frameworks that extract hand landmarks provide a structured representation for recognizing both static gestures and dynamic motion-based gestures. Despite these advances, gesture recognition systems remain sensitive to environmental factors such as lighting, camera angle, and user posture, highlighting the importance of careful system design and gesture selection [3].

2.3 MULTIMODAL INTERACTION SYSTEM

Multimodal interaction systems combine multiple input modalities, such as voice and gesture, to improve robustness and usability. Research shows that multimodal

systems reduce ambiguity by allowing users to express intent through complementary channels. Voice input is effective for semantic commands, while gestures are well suited for spatial or navigational control. However, multimodal systems introduce challenges related to synchronization, conflict resolution, and real-time performance. As a result, many studies emphasize modular architectures that allow each modality to operate independently while contributing to a unified control pipeline [2].

2.4 RESEARCH GAP AND MOTIVATION

Although significant research exists in voice interaction, gesture recognition, and multimodal systems, many existing solutions are either highly constrained or not designed for transparent evaluation. Commercial systems prioritize user experience over configurability, while research prototypes often lack integration with real-world applications. There is a clear need for a flexible, modular multimodal system that operates on consumer hardware, supports real-time interaction, and provides clear feedback and logging. This project addresses this gap by integrating voice and gesture recognition into a unified, research-oriented human–computer interaction system.

CHAPTER THREE

REALISTIC CONSTRAINTS

The development and deployment of the multimodal human–computer interaction system were subject to several realistic constraints. These constraints influenced design decisions, technology selection, system architecture, and evaluation methodology. Understanding these limitations is essential for assessing the feasibility, scalability, and real-world applicability of the proposed system.

3.1 TECHNICAL CONSTRAINTS

Several technical constraints shaped the implementation of the system.

Firstly, real-time performance requirements imposed limits on model complexity and processing pipelines. Gesture recognition relies on continuous webcam input and real-time hand tracking, while voice recognition requires low-latency wake word detection and transcription. Running both pipelines simultaneously on consumer-grade hardware necessitated lightweight models and efficient processing to maintain responsiveness.

Secondly, hardware dependency presented a constraint. The system relies on a functional webcam and microphone, and recognition accuracy is sensitive to camera resolution, lighting conditions, microphone quality, and background noise. Performance can degrade in low-light environments, noisy rooms, or when the user moves outside the camera’s field of view.

Thirdly, platform dependency restricted the system to macOS during development. Several system actions (such as window management, media control, and brightness adjustment) are implemented using macOS-specific APIs and system commands. As a result, the current implementation is not directly portable to other operating systems without additional platform-specific adaptations.

Finally, external service dependency introduces reliability constraints. Voice transcription and natural language understanding rely on third-party services (e.g., AssemblyAI and GPT-based APIs). Network latency, API rate limits, or service outages can temporarily affect system performance.

3.2 ECONOMIC CONSTRAINTS

Economic considerations influenced both development and deployment decisions.

The system was designed and tested using **consumer-grade hardware**, avoiding specialized sensors or dedicated depth cameras. This constraint ensured affordability and accessibility but limited the precision of gesture tracking compared to professional motion capture systems.

Additionally, cloud-based API usage costs imposed constraints on large-scale deployment. Speech-to-text and natural language processing services typically operate under usage-based pricing models. Continuous voice interaction or extended evaluation sessions can increase operational costs, making unrestricted long-term use economically challenging without further optimization or offline alternatives.

Budget constraints also affected dataset collection and evaluation scale. Extensive user studies, large annotated datasets, and professional usability testing were limited by time and financial resources, resulting in a prototype-focused evaluation rather than a full commercial deployment study.

3.3 ENVIRONMENTAL AND SUSTAINABILITY CONSTRAINTS

Environmental considerations were addressed primarily through software efficiency and hardware reuse.

The system was designed to run on existing personal devices (laptops with built-in webcams and microphones), reducing the need for additional hardware manufacturing and associated environmental impact. Lightweight models and

efficient runtime logic were preferred to minimize CPU usage and energy consumption during continuous operation.

However, reliance on cloud-based services introduces indirect environmental costs related to data transmission and server infrastructure. While these impacts are outside the direct control of the system, they represent a sustainability consideration for future large-scale deployments.

3.4 ETHICAL AND PRIVACY CONSTRAINTS

Ethical and privacy considerations were central to the system’s design.

The system processes audio and video data, which are inherently sensitive. Although gesture recognition operates on live video streams, no raw video frames are permanently stored by default. Similarly, voice data is processed transiently for transcription, and recordings are not retained unless explicitly enabled by the user.

User consent is implicitly required before activating microphone or camera access, and the interface provides clear visual indicators when sensors are active. Configuration options allow users to disable gesture detection, audio recording, and snapshot saving, reinforcing user control over personal data.

Nevertheless, the use of third-party transcription and language models introduces ethical concerns related to data handling outside the local system. While industry-standard APIs are used, future work would benefit from fully offline alternatives to further reduce privacy risks.

3.5 HEALTH AND SAFETY CONSTRAINTS

Health and safety constraints were considered in both physical and cognitive dimensions.

Prolonged gesture-based interaction may lead to physical fatigue, particularly when gestures require sustained arm or hand movement. To mitigate this, gestures were designed to be simple, short, and limited in number. The system is intended for intermittent interaction rather than continuous, high-intensity use.

From a cognitive perspective, the system aims to reduce mental workload by allowing natural language commands and intuitive gestures. However, misrecognition or delayed feedback could cause user frustration if not properly managed. Clear visual feedback and logs were included to maintain transparency and reduce uncertainty during interaction.

3.6 LEGAL AND REGULATORY CONSTRAINTS

The system operates within several legal and regulatory boundaries.

Access to microphone and camera devices is governed by operating system permissions, requiring explicit user approval. Failure to obtain or manage these permissions can prevent system operation. Additionally, the use of third-party APIs is subject to their respective terms of service, including acceptable use policies and data handling regulations.

Depending on deployment context, data protection regulations such as **GDPR** or similar privacy laws may apply, particularly if user data is stored, transmitted, or analyzed. Although the current prototype minimizes data retention, future extensions would require formal compliance assessments and user data protection mechanisms

CHAPTER FOUR

ARCHITECTURE

The multimodal human–computer interaction system is structured as a layered, modular architecture that separates user interface concerns from core recognition logic and operating system control. This design improves maintainability, extensibility, and cross-platform compatibility. At a high level, the system consists of three primary layers: the Electron desktop layer, the React-based front-end interface, and the Python-based backend engine. Each layer communicates through clearly defined boundaries, ensuring loose coupling and reliable data flow.

4.1 ELECTRON LAYER

The electron directory acts as the desktop application container and system orchestrator. Electron provides the bridge between the graphical user interface and the Python backend, allowing the application to behave like a native desktop program.

- **main.js** is the entry point of the desktop application. It is responsible for creating the application window and spawning the Python backend as a **child process**. Depending on user interaction (e.g., starting voice, gesture, or both), main.js launches the appropriate Python module and listens to its standard output and error streams.
- **preload.js** serves as a secure communication layer between the Electron main process and the front-end. It exposes a controlled API to the user interface using Electron’s contextBridge, preventing direct access to system-level APIs from the browser context.

Through this layer, Electron enables bidirectional communication: user actions from the UI trigger backend execution, and backend logs and events are forwarded back to the interface for display.

4.2 FRONT-END INTERFACE

The ui directory contains the React-based front-end, built using modern JavaScript tooling (React + Vite). This layer is responsible for all user-facing interaction, visualization, and configuration.

- The Dashboard page allows users to start the system in three modes: voice-only, gesture-only, or combined multimodal operation. It also displays live status indicators for microphone and camera activity.
- The History page reads stored log data and presents a chronological view of past voice commands, recognized gestures, and executed system actions.
- The Settings page enables users to configure runtime behavior such as auto-start options, device selection (microphone and webcam), and logging preferences.
- The Tutorial page provides guidance on supported gestures and voice commands, ensuring usability during demonstrations or user studies.

The front-end does not perform recognition itself. Instead, it delegates all processing to the backend, receiving feedback via Electron IPC and displaying it in real time.

4.3 BACKEND ENGINE

The engine directory contains the core intelligence of the system, implemented entirely in Python. It is divided into specialized submodules that handle voice recognition, gesture recognition, and action execution.

4.3.1 VOICE MODULE

This submodule implements the complete voice interaction pipeline. It continuously listens to the microphone, detects a wake word using a lightweight on-device model, records spoken commands, and sends the audio to an external speech-to-text service. The resulting transcription is processed by a natural language understanding (NLU) component that converts free-form speech into structured intents. These intents are

then forwarded to the dispatcher for execution. All recognized commands and system responses are logged for later review.

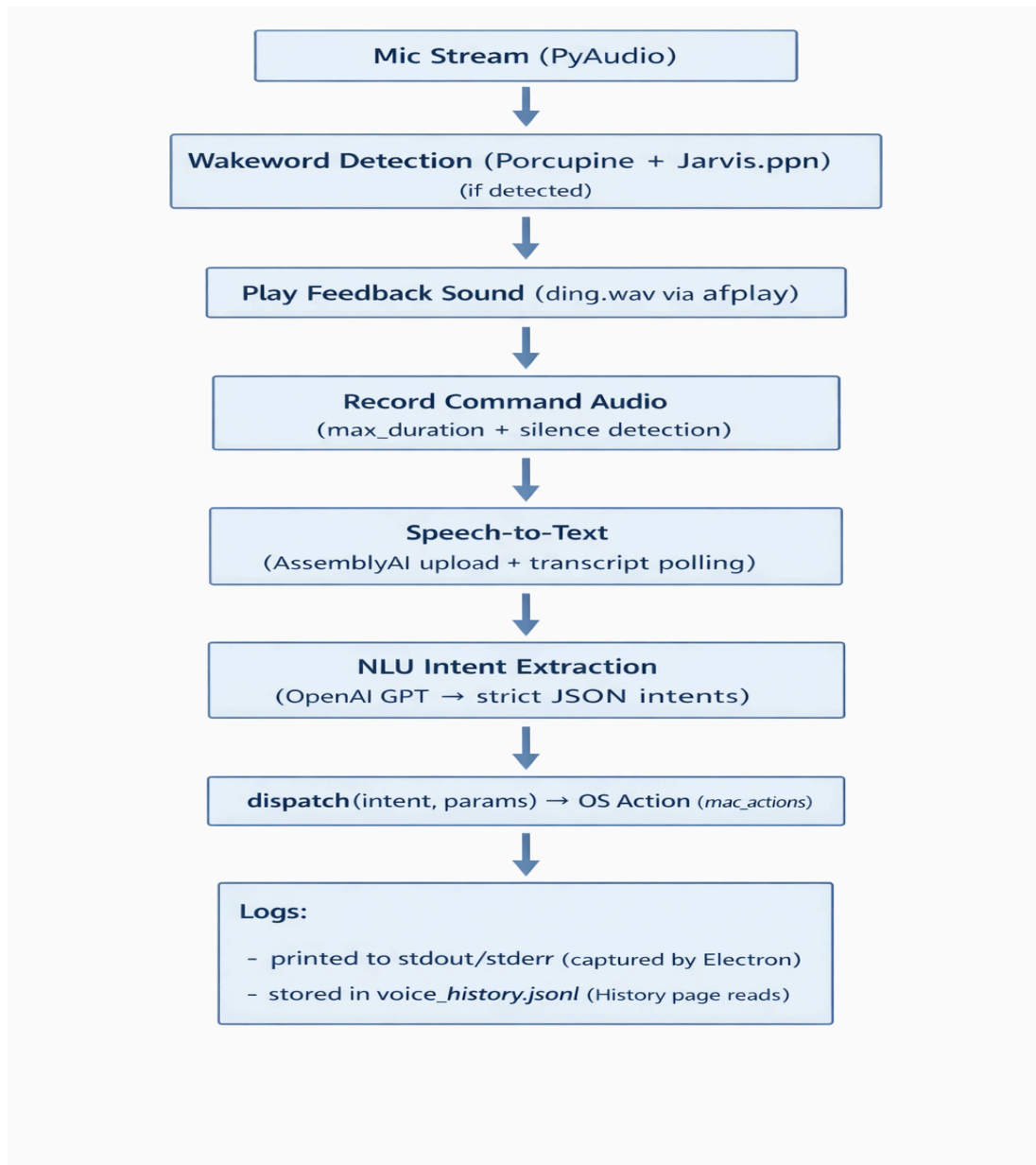


Fig1. voice module architecture.

4.3.2 GESTURE MODULE

The gesture subsystem processes live webcam frames using computer vision techniques. Hand landmarks are extracted using MediaPipe, normalized, and passed to classifiers that recognize both static hand poses and dynamic motion-based gestures. Temporal filtering is applied to ensure stability and reduce false positives. Once a gesture is confidently recognized, it is mapped to a system action and dispatched for execution.

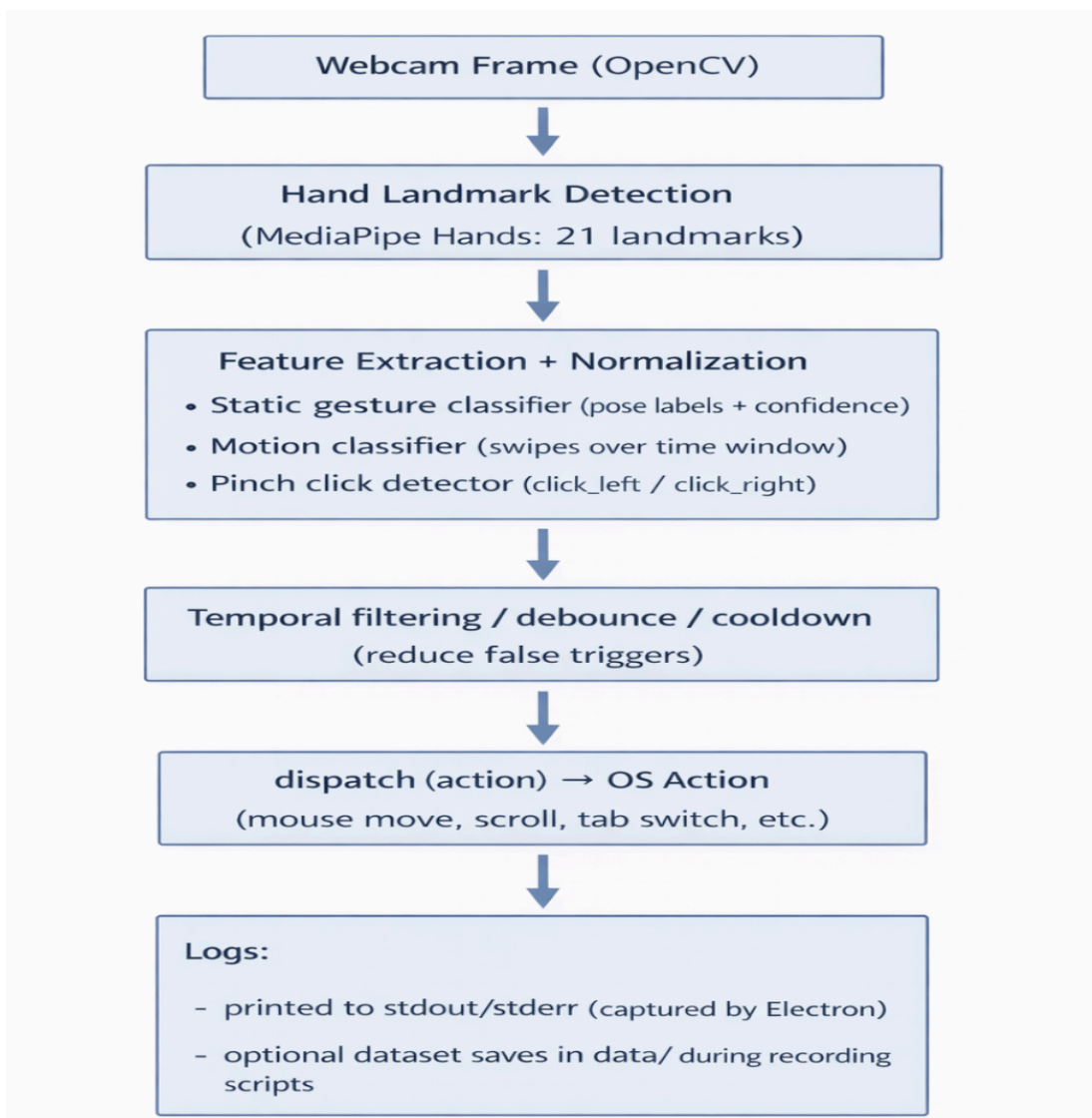


Fig3. gesture module architecture

4.3.3 DISPATCHER MODULE

The dispatcher acts as a central routing layer between recognition modules and operating system control. Both voice and gesture pipelines invoke a common `dispatch()` interface, ensuring consistent behavior regardless of input modality. Platform-specific implementations (macOS or Windows) translate abstract actions, such as “increase volume” or “switch tab”, into concrete system-level commands.

4.4 ORCHESTRATION AND CONTEXT

The backend also includes orchestration scripts that allow voice and gesture pipelines to run independently or concurrently, depending on the selected mode. Shared context utilities ensure consistent state management when both modalities are active simultaneously.

4.5 OVERALL INTERACTION FLOW

In operation, the user interacts with the system through the front-end interface. User actions are forwarded to the Electron layer, which launches or controls the Python backend. The backend processes audio and video input, recognizes intents or gestures, executes system actions through the dispatcher, and emits logs. These logs flow back through Electron to the front-end, creating a closed feedback loop that allows the user to observe system behavior in real time.



Fig3. whole system architecture.

CHAPTER FIVE

SYSTEM TOOLS, TECHNOLOGIES, AND DATA COLLECTION

5.1 OVERVIEW

This chapter presents the tools, technologies, and methodologies used in the development of the multimodal human–computer interaction system. The system integrates voice recognition and hand gesture recognition to enable intuitive, real-time control of a computing device. To achieve accuracy, responsiveness, and portability, the project relies on lightweight libraries, modular system design, and carefully structured data collection processes.

The chapter is divided into three main parts. First, the software tools and development environment are discussed. Next, the libraries and frameworks used for voice and gesture recognition are explained. Finally, the data collection methodology for gesture recognition is detailed, including the process of recording, labeling, and storing gesture samples.

5.2 SOFTWARE TOOLS AND DEVELOPMENT

The system was developed primarily using Python, chosen for its extensive ecosystem of machine learning, computer vision, and audio-processing libraries, as well as its cross-platform compatibility. The development environment was structured to support modular execution of voice and gesture components while allowing both systems to run independently or simultaneously.

A virtual environment (venv) was used to manage dependencies and ensure version consistency across modules. Development and testing were performed on macOS, with explicit consideration given to Windows compatibility through platform-specific dispatcher modules.

Version control was handled using Git, enabling incremental development, testing, and rollback of features. The project repository was structured into logical modules, including voice processing, gesture recognition, action dispatching, and system integration. This modular architecture simplified debugging and allowed individual components to be tested independently before integration.

5.3 LIBRARIES AND FRAMEWORK USED

5.3.1 VOICE RECOGNITION AND PROCESSING LIBRARIES

The voice interaction component was designed as a hybrid system combining wake-word detection, speech-to-text transcription, natural language understanding, and text-to-speech feedback.

- **openai-whisper**

Used for high-accuracy speech-to-text transcription. Whisper's transformer-based architecture allows robust recognition across different accents and speaking styles while supporting offline processing.

- **SpeechRecognition**

Used as an interface layer for capturing microphone input and managing audio streams.

- **PyAudio**

Provides low-level access to audio hardware, enabling real-time microphone input and continuous listening.

- **pvporcupine**

Implemented for wake-word detection ("Jarvis"), allowing the system to remain idle until explicitly activated, reducing unnecessary computation.

- **rapidfuzz**

Used for fuzzy string matching during intent resolution, enabling flexible interpretation of spoken commands rather than rigid keyword matching.

- **OpenAI / AssemblyAI APIs**

Utilized for higher-level natural language understanding when required, allowing complex intent extraction and contextual command interpretation.

- **pyttsx3**

Provides offline text-to-speech functionality, enabling verbal system feedback without reliance on external services.

- **FastAPI and Uvicorn**

Used to expose system functionality through lightweight APIs, allowing clean separation between recognition logic and application control.

5.3.2 Gesture Recognition Libraries

The gesture recognition system was designed to prioritize real-time responsiveness and stability over computational complexity.

- **MediaPipe Hands**

Used for real-time hand tracking and landmark extraction. MediaPipe provides 21 three-dimensional landmarks per detected hand, enabling precise representation of hand posture and movement.

- **OpenCV (opencv-python)**

Handles webcam access, frame processing, visualization, and real-time feedback during gesture capture and execution.

- **NumPy**

Used for numerical operations, including normalization of landmark vectors, smoothing calculations, and gesture feature representation. Custom logic was built on top of these libraries to classify gestures, smooth cursor movement using exponential moving averages (EMA), and suppress jitter during minor hand movements.

5.4 GESTURE DATA COLLECTION METHODOLOGY

Accurate gesture recognition requires representative and well-labeled data. Instead of relying on pre-existing datasets, a custom gesture dataset was created to ensure compatibility with the system's real-time constraints and intended gestures.

5.4.1 DATE CAPTURE PROCESS

Gesture data was collected using a webcam in a controlled real-time environment. A custom recording script was developed to allow manual, user-driven data capture. During recording:

- The webcam continuously streamed frames using OpenCV.
- MediaPipe Hands detected and tracked the user's hand.
- The user pressed the **r** key to capture a gesture sample at the desired moment.
- Each captured frame was converted into a **normalized landmark vector** representing the hand pose.

This approach allowed precise control over when samples were recorded, ensuring consistency and reducing noisy or ambiguous samples.

5.4.2 LABELLING AND STORAGE

Each recorded gesture was assigned a label (e.g., `move_cursor`, `scroll_up`, `scroll_down`) and appended directly to a JSON-based dataset. The dataset structure allowed new samples to be added incrementally without overwriting existing data.

This method supported iterative improvement of gesture accuracy, as new samples could be recorded whenever recognition performance degraded or new gestures were introduced. Over time, hundreds of samples were collected for frequently used gestures, improving classification reliability.

5.4.3 DATASET VALIDATION

After data collection, gestures were tested live within the gesture runtime environment. This allowed immediate verification of recognition accuracy and responsiveness. Samples that caused instability or misclassification were discarded or replaced with improved recordings, ensuring the dataset remained clean and effective.

5.5 TOOLS USED FOR THE FRONT-END APPLICATION

The desktop interface for the multimodal system was implemented using Electron, which enables a cross-platform desktop application using web technologies while still allowing access to system-level features. The user interface itself was developed in React, providing a component-based structure for building the dashboard layout, status panels, and activity log display.

For styling and layout consistency, the UI uses Tailwind CSS, allowing rapid development of a clean, responsive interface. Icons and visual indicators (e.g., microphone, camera, gesture status) were implemented using the Lucide React icon library.

Communication between the Electron UI and the Python runtime was designed using Electron's IPC (Inter-Process Communication) layer, with a preload script exposing a secure API to the React frontend via contextBridge. The backend runtime is launched using Node's `child_process` utilities, enabling the UI to start or stop Python execution modes (voice-only, gesture-only, or both) while streaming runtime logs directly into the dashboard.

CHAPTER SIX

VOICE SYSTEM IMPLEMENTATION

6.1 OVERVIEW OF THE VOICE INTERACTION PIPELINE

This chapter describes the detailed implementation of the voice interaction subsystem developed for the multimodal human–computer interaction system. The objective of this subsystem is to enable natural, hands-free control of the operating system using spoken language. Unlike traditional command-based systems, the implemented solution supports continuous listening, wake word activation, free-form speech commands, intent extraction, and parameterized action execution.

The voice subsystem is designed as a pipeline architecture, where each stage performs a well-defined role and passes its output to the next stage. This modular approach improves maintainability, scalability, and clarity, while also allowing individual components to be tested and refined independently. The main stages of the pipeline include:

1. Wake word detection
2. Speech recording and preprocessing
3. Speech-to-text transcription
4. Natural language understanding (NLU)
5. Action dispatching and execution

Each of these stages is implemented using a dedicated Python module, which are explained in detail in the following sections.

6.2 WAKE WORD DETECTION AND RUNTIME CONTROL

```
def main():
    access_key = load_access_key()

    # Use custom Jarvis wakeword file
    base_dir = Path(__file__).resolve().parent
    keyword_path = base_dir / "wakewords" / "Jarvis.ppn" # Make sure name/case matches
    if not keyword_path.exists():
        raise FileNotFoundError(f"Wake word file not found: {keyword_path}")

    porcupine = pporcupine.create(
        access_key=access_key,
        keyword_paths=[str(keyword_path)],
    )

    pa = pyaudio.PyAudio()
    audio_stream = pa.open(
        rate=porcupine.sample_rate,
        channels=1,
        format=pyaudio.paInt16,
        input=True,
        frames_per_buffer=porcupine.frame_length,
    )

    print("Voice system ready.")
    print("Say the wake word: 'jarvis'") # Displays this as soon as the program is active |

    try:
        while True:
            pcm_bytes = audio_stream.read(porcupine.frame_length, exception_on_overflow=False)
            pcm = struct.unpack_from("h" * porcupine.frame_length, pcm_bytes)

            keyword_index = porcupine.process(pcm)
            if keyword_index >= 0:
                print("Wake word detected.")
```

Fig4. voice_main_wake.py

The file `voice_main_wake.py` serves as the primary runtime controller for the voice subsystem. It is responsible for initializing all required audio components, continuously monitoring microphone input, detecting the wake word, and coordinating the downstream processing of spoken commands.

6.2.1 CONTINUOUS AUDIO STREAMING

The system uses the **PyAudio** library to open a low-level audio stream connected directly to the system microphone. Audio is captured as short, fixed-size frames rather than large recordings. This approach minimizes latency and allows real-time processing.

The frame size and sample rate are dictated by the wake word engine to ensure compatibility. Each frame is read synchronously and processed immediately, allowing the system to remain responsive even during extended periods of idle listening.

6.2.2 WAKE WORD DETECTION USING PICOVOICE PORCUPINE

Wake word detection is implemented using Picovoice Porcupine [8], a highly optimized keyword spotting engine designed for embedded and real-time systems. A custom-trained wake word model (Jarvis.ppn) is loaded at runtime using a Picovoice access key stored securely in a local configuration file.

Each incoming audio frame is converted into a numeric pulse-code modulation (PCM) representation and passed to the Porcupine engine. When the engine identifies a match with the wake word model, it returns a non-negative index, signalling successful detection [8].

This approach ensures that speech commands are only processed after explicit user activation, reducing false positives and unnecessary computation.

6.2.3 USER FEEDBACK AND STATE TRANSITION

Upon detecting the wake word, the system immediately plays an audible confirmation sound (ding.wav) using the macOS afplay utility. The sound is played asynchronously to avoid blocking the microphone stream.

At this point, the system transitions from idle listening mode to command capture mode, where it begins recording the user's spoken instruction.

6.3 SPEECH RECORDING AND VOICE ACTIVITY DETECTION

6.3.1 CONTROLLED COMMAND RECORDING

Rather than recording a fixed-length audio clip, the system employs a hybrid recording strategy combining a maximum duration constraint with silence-based termination. This is implemented in the `record_command` function.

Audio frames are continuously appended to an in-memory buffer until one of the following conditions is met:

- The maximum recording duration is reached
- Prolonged silence is detected after speech has begun

This strategy balances flexibility and efficiency, allowing users to speak naturally without requiring manual start or stop commands.

6.3.2 ENERGY-BASED SILENCE DETECTION

Silence detection is performed using a simple but effective energy-based method. For each audio frame, the average signal amplitude is computed. Frames with energy below a predefined threshold are treated as silence.

If silence persists beyond a configurable timeout after speech has been detected, recording is terminated early. This reduces unnecessary transcription of background noise and improves responsiveness.

6.3.3 IN-MEMORY AUDIO ENCODING

Once recording is complete, the captured frames are encoded into a standard WAV format using an in-memory buffer. This avoids writing temporary files to disk, reducing latency and minimizing resource usage.

The resulting byte stream is then passed directly to the transcription module.

6.4 SPEECH-TO-TEXT TRANSCRIPTION

```
def upload_audio(audio_bytes: bytes) -> str:
    """
    Upload raw audio bytes to AssemblyAI and return the audio_url.
    """
    headers = {
        "authorization": ASSEMBLYAI_API_KEY
    }

    response = requests.post(f"{BASE_URL}/upload", headers=headers, data=audio_bytes)

    if not response.ok:
        print("[AssemblyAI upload error]", response.status_code, response.text)
        response.raise_for_status()

    upload_url = response.json().get("upload_url")
    if not upload_url:
        raise RuntimeError("No upload_url returned from AssemblyAI.")
    return upload_url

def transcribe_audio_bytes(audio_bytes: bytes, timeout_sec: int = 30) -> str:
    """
    High-level helper:
    - uploads audio bytes
    - creates a transcript
    - polls until completed or error
    - returns the transcribed text
    """
    audio_url = upload_audio(audio_bytes)

    headers = {
        "authorization": ASSEMBLYAI_API_KEY,
        "content-type": "application/json"
    }

    # 1) Create transcript job
    body = {
        "audio_url": audio_url,
        "language_code": "en", # generic English, supports many accents
        "punctuate": True
    }

    resp = requests.post(f"{BASE_URL}/transcript", json=body, headers=headers)
    resp.raise_for_status()
```

Fig5. assemblyai_client.py

The transcription stage is implemented in `assemblyai_client.py`, which provides a clean abstraction for interacting with the AssemblyAI speech recognition service.

6.4.1 API KEY MANAGEMENT AND SECURITY

The AssemblyAI API key is loaded from a local configuration file located at the project root. This approach avoids hardcoding sensitive credentials into the source code and supports secure key rotation.

6.4.2 AUDIO UPLOAD PROCESS

The recorded audio bytes are uploaded directly to AssemblyAI's servers using an authenticated HTTP request. Upon successful upload, the service returns a temporary audio URL, which is then used to create a transcription job [5][6].

6.4.3 TRANSCRIPTION JOB LIFECYCLE

After creating a transcription job, the system enters a polling loop where it periodically checks the job's status. This continues until one of the following outcomes occurs:

- The transcription completes successfully
- An error is reported by the service
- A timeout threshold is exceeded

Once completed, the transcribed text is returned to the main runtime controller.

This asynchronous polling approach ensures robustness and allows the system to handle variable network latency gracefully.

6.5 NATURAL LANGUAGE UNDERSTANDING

```
def _llm_intents_with_params(text: str) -> List[Dict[str, Any]]:
    """
    Ask GPT-4.1 to parse the user text into a list of
    {intent, confidence, params}.
    Returns [] on any error or if no intent fits.
    """
    if not LLM_ENABLED or not text:
        return []

    system_prompt = (
        "You are the NLU module for a desktop voice assistant on macOS.\n"
        "Your job is to map the user's natural language command into "\n"
        "a small set of high-level intents plus parameters.\n\n"
        "You MUST respond with STRICT JSON, with NO explanations.\n"
        "Respond in ONE of these formats only:\n"
        "1) A bare JSON array:\n"
        "   [\n"
        "     {\n"
        "       \"intent\": \"volume_up\", \"confidence\": 1.0, \"params\": {},\n"
        "       \"intent\": \"search\", \"confidence\": 0.9, \"\n"
        "       \"params\": {\"query\": \"who is cristiano ronaldo\"}}\n"
        "     ]\n"
        "OR\n"
        "2) An object with an 'intents' field:\n"
        "   {\n"
        "     \"intents\": [ ...same as above... ]\n"
        "   }\n"
        "Each element MUST have exactly:\n"
        "  - intent: one of the allowed intents\n"
        "  - confidence: number between 0.0 and 1.0\n"
        "  - params: an object with any extra info (may be empty)\n"
        f"Allowed intents are: {', '.join(INTENT_WHITELIST)}.\n"
        "Guidance:\n"
        "- For simple device control like volume/brightness/scroll, "\n"
        "  choose the obvious intent, e.g. 'volume_up', 'brightness_down', 'scroll_up'.\n"
        "- For opening apps like 'open whatsapp' or 'launch spotify', "\n"
        "  use 'open_app_generic' and set params.app_name to a clean macOS app name, "\n"
        "  e.g. 'WhatsApp', 'Spotify', 'Visual Studio Code'.\n"
        "- For closing apps ('close this app', 'quit chrome'), use 'close_app_generic'.\n"
        "- For questions like 'what is the time' or 'what's today's date', "\n"
        "  use 'what_time' or 'what_date'."
    )
```

Fig6. nlu.py

The nlu.py module is responsible for converting raw transcribed text into structured, machine-readable intents that can be executed by the system[6].

6.5.1 USE OF A LARGE LANGUAGE MODEL

The NLU component leverages **GPT-4.1** via the OpenAI API to perform semantic interpretation of user commands. Instead of relying on brittle keyword matching or rule-based parsing, the language model is prompted to return structured JSON output containing:

- A recognized intent
- A confidence score
- Any extracted parameters

This enables the system to handle natural phrasing variations and complex multi-part commands.

6.5.2 STRICT OUTPUT ENFORCEMENT AND VALIDATION

To ensure reliability, the model is instructed to output **strict JSON only**, without explanatory text. The returned data is parsed and validated against a predefined whitelist of supported intents.

Only intents with confidence scores above a configurable threshold are accepted. This prevents ambiguous or low-confidence interpretations from triggering unintended actions.

6.5.3 MULTI-INTENT COMMAND SUPPORT

The system supports commands that map to multiple actions, such as opening an application and performing a search within it. These intents are executed sequentially, preserving logical order.

6.6 ACTION DISPATCHING AND EXECUTION

```
# SCREENSHOT
elif intent == "capture_screen":
    run_osascript(
        'tell application "System Events" to key code 20 using {command down, shift down}'
    )

# LOCK SCREEN
elif intent == "lock_screen":
    run_osascript(
        'tell application "System Events" to keystroke "q" using {control down, command down}'
    )

# TIMER / ALARM
elif intent == "set_timer":
    # params should include: seconds (int) OR minutes (int)
    seconds = int(params.get("seconds") or 0)
    minutes = int(params.get("minutes") or 0)
    total = seconds + minutes * 60
    if total <= 0:
        say("How long should I set the timer for?")
        return
    SCHED.set_timer(total, label="Timer")
    say(f"Timer set for {total//60} minutes." if total >= 60 else f"Timer set for {total} seconds.")

elif intent == "set_alarm":
    # params should include hour, minute, ampm (AM/PM)
    from datetime import datetime, timedelta
    hour = int(params.get("hour"))
    minute = int(params.get("minute", 0))
```

Fig7. dispatcher

Validated intents are passed to the dispatcher layer, which maps high-level intents to concrete operating system actions. This separation ensures that the NLU module remains independent of platform-specific logic.

The dispatcher executes each intent using system-level commands, enabling interaction with applications, system settings, and user interface elements.

6.7 SYSTEM INTEGRATION AND CONTROL FLOW

The voice interaction pipeline operates as a tightly integrated system with clear boundaries between stages. The complete flow is as follows:

1. Continuous microphone monitoring
2. Wake word detection
3. Speech recording with silence detection

4. Cloud-based transcription
5. Intent extraction and validation
6. Action execution

Each component communicates through well-defined interfaces, enabling reliable real-time performance and simplifying debugging and extension.

6.8 CHAPTER SUMMARY

This chapter presented a detailed explanation of the voice interaction subsystem, covering its architecture, implementation, and integration. By combining efficient real-time audio processing, cloud-based speech recognition, and large language model-driven intent parsing, the system achieves robust and flexible voice control.

The modular design described in this chapter provides a strong foundation for extending the system to additional modalities, such as gesture recognition, which is explored in the next chapter.

CHAPTER SEVEN

GESTURE RECOGNITION SYSTEM IMPLEMENTATION

7.1 OVERVIEW OF GESTURE IMPLEMENTATION

This chapter presents the design and implementation of the gesture recognition subsystem developed for the multimodal human–computer interaction system. The goal of this subsystem is to enable users to interact with the operating system using natural hand gestures, captured through a standard webcam and interpreted in real time.

Unlike traditional vision-based systems that rely on pre-trained datasets or fixed gesture vocabularies, the implemented solution emphasizes custom gesture creation, real-time classification, and tight integration with system-level actions. The system allows gestures to be recorded, trained, validated, and executed within a unified pipeline [4].

The gesture subsystem is implemented entirely in Python and is structured into modular components responsible for:

- Hand landmark detection
- Gesture data recording
- Feature extraction and classification
- Real-time gesture recognition
- Motion-based gestures and cursor control
- Action dispatching

This modular architecture ensures clarity, extensibility, and separation of concerns, which is essential for both system robustness and academic evaluation.

7.2 HAND TRACKING AND LANDMARK EXTRACTION

7.2.1 MEDIA PIPE-BASED HAND DETECTION

At the core of the gesture system is MediaPipe Hands [7], a real-time hand tracking framework that detects and tracks 21 anatomical landmarks per hand. These landmarks represent joints and fingertip positions and form the raw input to the gesture recognition pipeline .

MediaPipe operates directly on RGB frames captured from the webcam using OpenCV. Each frame is processed independently, allowing the system to operate at interactive frame rates without requiring GPU acceleration[13].

7.3 GESTURE DATA RECORDING

```
def main():
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("Could not open webcam.")
        return

    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

    hands = mp_hands.Hands(
        static_image_mode=False,
        max_num_hands=1,
        min_detection_confidence=0.6,
        min_tracking_confidence=0.6,
    )

    print("Custom gesture recorder")
    print("Press 'r' to record current hand pose as a sample.")
    print("Press 'q' or ESC to quit.")
```

Fig8. file gesture_record.py

The file gesture_record.py is responsible for collecting custom gesture samples from the user. Rather than relying on pre-existing datasets, this approach allows the system

to be trained using gestures performed by the intended user, improving personalization and robustness.

7.3.1 CONTROLLED DATA CAPTURE

Gesture recording is performed using live webcam input. When the user presses a designated key (e.g., r), the system captures the current hand landmarks detected by MediaPipe and stores them as a training sample.

Each sample consists of the full set of landmark coordinates, which preserves the spatial relationships between fingers and joints.

7.3.2 DATASET ORGANIZATION

Recorded samples are stored in a structured format, grouped by gesture label. This enables straightforward loading, retraining, and validation during later stages.

By explicitly controlling when samples are captured, the system avoids noisy or ambiguous data, ensuring a higher-quality training dataset.

7.4 GESTURE DEFINITION AND CONFIGURATION

```
"move_cursor": [  
  [  
    0.0,  
    0.0,  
    -0.304113528009369,  
    -0.304383400336976,  
    -0.4743666216945104,  
    -0.70241751146181,  
    -0.599134113166273,  
    -0.9540676083289789,  
    -0.7581753902441158,  
    -1.105166033139344,  
    -0.182223978433712,  
  ],  
]
```

Fig9. custom_gestures.json.

The gesture system supports user-defined gestures through a configuration-driven design. Gesture metadata, including gesture names and mappings, are stored in custom_gestures.json.

This file acts as a single source of truth for all available gestures, decoupling gesture definitions from implementation logic.

The accompanying Python module, `custom_gestures.py`, loads and validates this configuration, making gesture definitions accessible throughout the system.

This approach allows new gestures to be added without modifying core classification or runtime code, improving extensibility and maintainability.

7.5 FEATURE EXTRACTION AND GESTURE CLASSIFICATION

7.5.1 STATIC GESTURE CLASSIFICATION

```
# Mediapipe hand landmark indices:
# Thumb: 4 (TIP)
# Index: 8 (TIP), 6 (PIP)
# Middle: 12 (TIP), 10 (PIP)
# Ring: 16 (TIP), 14 (PIP)
# Pinky: 20 (TIP), 18 (PIP)

def _is_finger_extended(lm, tip_idx: int, pip_idx: int, margin: float = 0.04) -> bool:
    """
    Decide if a finger is extended by comparing TIP.y and PIP.y.
    In Mediapipe, y = 0 at top, increases downward.

    So: TIP.y < PIP.y - margin -> finger pointing up (extended).
    """
    tip_y = lm[tip_idx].y
    pip_y = lm[pip_idx].y
    return tip_y < (pip_y - margin)

def classify_static_gesture(hand_landmarks, image_width: int, image_height: int) -> Optional[str]:
    """
    Classify a static gesture from Mediapipe HandLandmarks.

    We ignore width/height here because we use normalized coordinates,
    but we keep them in the signature for future use if needed.
    """

    lm = hand_landmarks.landmark # 21 landmarks, normalized

    # Check extension of four main fingers (ignore thumb for now)
    index_ext = _is_finger_extended(lm, tip_idx=8, pip_idx=6)
    middle_ext = _is_finger_extended(lm, tip_idx=12, pip_idx=10)
    ring_ext = _is_finger_extended(lm, tip_idx=16, pip_idx=14)
    pinky_ext = _is_finger_extended(lm, tip_idx=20, pip_idx=18)
```

Fig10. `gesture_classifier.py`

The `gesture_classifier.py` module implements classification for static hand poses, such as open palm, fist, or custom-defined shapes.

Landmark coordinates are transformed into feature vectors that describe relative joint positions rather than absolute screen coordinates. This normalization step ensures invariance to hand size, camera distance, and position within the frame.

The classifier outputs a gesture label and confidence score, which are used to determine whether the gesture is valid and stable enough to trigger an action.

7.5.2 CUSTOM GESTURE CLASSIFICATION

```
# CLASSIFIER

def classify_custom_static(
    hand_landmarks: mp_hands.HandLandmark,
    min_score: float = 0.97,
    other_margin: float = 0.01,
) -> Tuple[Optional[str], float]:
    """
    Confidence-first custom gesture classifier.

    Rules:
    - If best_score >= min_score -> RETURN label
    - Otherwise -> RETURN None
    """

    gestures = _load_custom_gestures()
    if not gestures:
        return None, -1.0

    live_vec = _landmarks_to_vector(hand_landmarks)

    best_label: Optional[str] = None
    best_score: float = -1.0

    second_best: float = -1.0

    for label, samples in gestures.items():
        for sample in samples:
            score = _cosine_similarity(
                live_vec,
                np.asarray(sample, dtype=np.float32),
```

Fig11. custom_gesture_classifier.py

This module extends the static classifier to support user-trained gestures. It loads recorded samples and applies a supervised classification approach to match live hand poses against stored gesture templates.

By supporting custom training, the system avoids dependence on large external datasets and enables rapid experimentation with new gestures.

7.6 REAL-TIME GESTURE RECOGNITION

The real-time execution of the gesture system is handled by `gesture_runtime.py` which is the main file for mapping the actions detected and the intents in real time. This file acts as the main event loop for gesture recognition.

7.6.1 FRAME PROCESSING LOOP

Each iteration of the loop performs the following steps:

1. Capture a frame from the webcam
2. Detect hand landmarks using MediaPipe
3. Extract features from landmarks
4. Classify the gesture
5. Apply temporal smoothing
6. Dispatch the corresponding action

This pipeline is executed continuously, allowing gestures to be recognized and acted upon in near real time.

7.6.2 STABILITY AND TEMPORAL FILTERING

To prevent false positives caused by momentary hand movements, the system applies temporal filtering. A gesture must be detected consistently across multiple consecutive frames before it is accepted as valid.

This design choice significantly improves usability and reduces accidental triggers.

7.7 MOTION-BASED GESTURE AND CURSOR CONTROL

7.7.1 MOTION GESTURE CLASSIFICATION

```
@dataclass
class MotionResult:
    label: Optional[str] # "swipe_left", "swipe_right", "swipe_up", "swipe_down", or None
    confidence: float # 0..1

class MotionGestureClassifier:
    """
    Rule-based dynamic gesture detector using wrist trajectory over a rolling window.
    - Feed it each frame's hand_landmarks (or None).
    - Call update(...) -> MotionResult
    """

    def __init__(
        self,
        window_sec: float = 0.7,
        min_frames: int = 10,
        min_move: float = 0.18, # normalized movement threshold (tune)
        dominance: float = 1.4, # axis dominance ratio (tune)
        cooldown_sec: float = 0.6, # prevent double triggers
    ):
        self.window_sec = window_sec
        self.min_frames = min_frames
        self.min_move = min_move
        self.dominance = dominance
        self.cooldown_sec = cooldown_sec

        self.points: Deque[Tuple[float, float, float]] = deque()
        self.last_fire_time = 0.0
```

Fig12. motion_gesture_classifier.py

Not all gestures are static hand poses. Some interactions, such as swipes or directional movements, depend on motion over time.

This module analyzes changes in hand position across successive frames to detect directional gestures, such as left-to-right or up-and-down movements.

7.7.2 CURSOR MOVEMENT AND CLICK DETECTION

The files `record_move_cursor.py` and `click_motion.py` implement cursor control and click gestures. These modules map hand movement trajectories to mouse actions, enabling pointer control without physical input devices.

Motion thresholds and smoothing are applied to ensure precise and stable control.

7.8 GESTURE-CONTROLLED ACTIONS

The `gesture_control.py` module acts as the bridge between gesture recognition and system actions. It maps recognized gestures to high-level intents, which are then dispatched using the same action execution framework as the voice subsystem.

This unified action layer ensures consistency across modalities and simplifies integration within the multimodal system.

7.9 GESTURES AND MOTIONS

The gestures and motion gestures in total are 8 different straight forward and distinct actions. The first one is the move cursor and shown in Fig17. When your gesture is in frame it automatically activates the cursor and you can move it around in real time. Same as the screenshot gesture in Fig19 , scroll up gesture in Fig21 scroll down gesture in Fig20.

The rest of the gestures are motion gestures, the left click gesture in Fig18 works by double tapping your index finger and your thumb. The right click works by double tapping your ring finger and your thumb shown in Fig19.

The swipe left and right gestures work almost the same but the slight difference makes it distinct and avoids contradiction. The swipe left gesture works by putting up the gesture in Fig22 and swiping left and the right swipe works by by putting up the gesture in Fig23 and swiping right

CHAPTER EIGHT

APPLICATION INTERFACE DESIGN AND INTEGRATION

8.1 OVERVIEW OF THE APPLICATION INTERFACE

This chapter presents the design and implementation of the desktop application interface developed for the Multimodal Human–Computer Interaction system [10[11]]. While earlier chapters focused on the underlying voice and gesture recognition pipelines, this chapter shifts attention to how these capabilities are exposed to users through an intuitive, interactive, and controllable interface.

The application acts as the central control hub of the system. It allows users to:

- Start and stop the assistant in different operating modes (voice-only, gesture-only, or combined multimodal mode),
- Observe real-time feedback from both recognition pipelines,
- Review historical logs of system activity,
- Configure devices and system behavior, and
- Learn how to interact with the system through a built-in tutorial.

Rather than serving as a passive display, the interface actively manages the lifecycle of the Python-based recognition engines and reflects their internal state in real time.

8.2 FRONT-END TECHNOLOGY STACK

The application interface was implemented as a desktop application using web technologies, enabling rapid development while maintaining a modern and responsive user experience.

The primary technologies used are:

- **React:** Used to implement the user interface as a component-based architecture. React’s state management model allows the interface to dynamically reflect

system status such as microphone activity, gesture detection state, and recognition confidence values.

- **Electron**: Used to package the React application as a cross-platform desktop application. Electron provides access to native operating system features and enables communication with the Python backend.
- **HTML5 / CSS3**: Used for layout, styling, and responsive design.
- **JavaScript (ES6+)**: Used to manage application logic, event handling, and inter-process communication.

This combination allows the system to benefit from the flexibility of web-based UI development while still operating as a native desktop application capable of launching and controlling external processes.

8.3 APPLICATION ARCHITECTURE AND BACKEND INTEGRATION

The application follows a two-layer architecture:

1. Front-End Layer (Electron + React)
Responsible for user interaction, visualization, and system control [12].
2. Backend Layer (Python Engines)
Responsible for gesture recognition, voice recognition, natural language understanding, and action execution.

The integration between these layers is achieved through Electron's inter-process communication (IPC) mechanism. When the user interacts with the interface (for example, clicking "Start Voice"), the Electron main process spawns the appropriate Python module as a child process. Standard output and error streams from Python are continuously captured and forwarded back to the UI.

This architecture ensures:

- Clear separation of concerns,
- Independent development of recognition logic and UI,

- Robust process control (start, stop, restart),
- Real-time feedback without blocking the interface.

8.4 MAIN DASHBOARD DESIGN

8.4.1 DASHBOARD PURPOSE

The Dashboard is the primary interaction screen of the application. It is designed to provide real-time system control and feedback while keeping cognitive load low for the user.

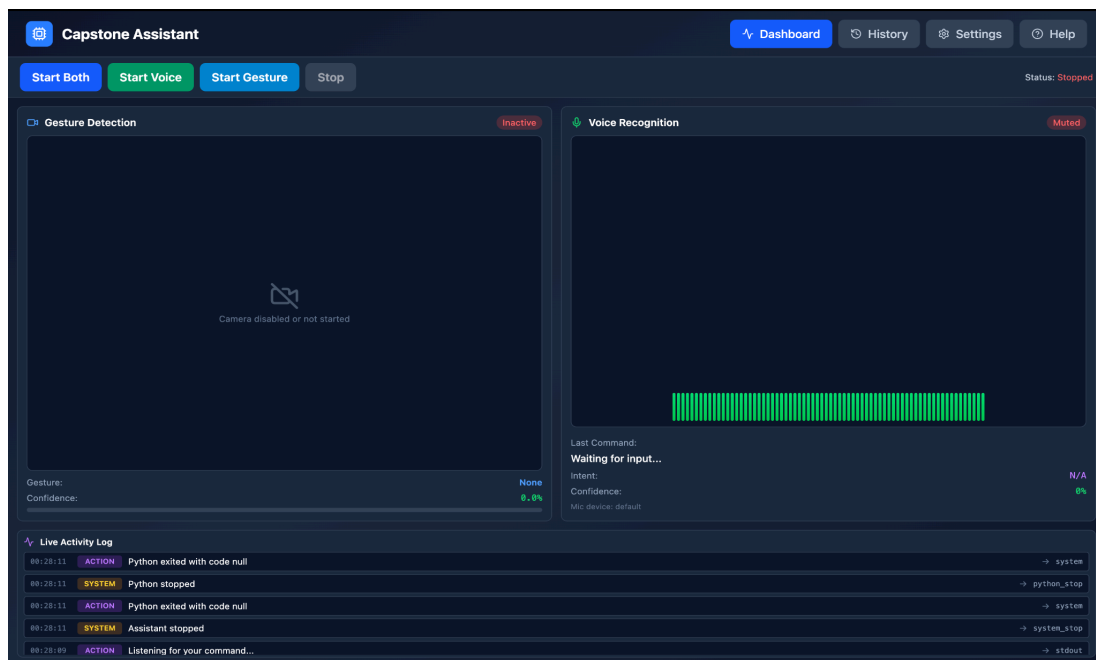


Fig13. Dashboard design

8.4.2 MODE CONTROL AND EXECUTION FLOW

At the top of the Dashboard, three primary control buttons are provided:

1. Start Voice
2. Start Gesture
3. Start Both

4. Stop

Each button directly corresponds to a backend execution mode:

- Voice-only mode launches the voice recognition engine.
- Gesture-only mode launches the gesture recognition runtime.
- Combined mode launches both pipelines simultaneously.

This explicit separation was intentionally chosen to:

- Simplify user understanding,
- Support controlled experiments and demonstrations,
- Avoid hidden dependencies between modalities.

When a mode is started, the interface:

1. Updates the visual system status,
2. Launches the corresponding Python process,
3. Begins receiving and displaying runtime logs.

Switching modes requires stopping the current process and starting a new one, ensuring that system state remains predictable and stable.

8.4.3 GESTURE DETECTION PANEL

The Gesture Detection panel displays the current state of the gesture recognition system.

Key elements include:

- A live camera preview (when gesture mode is active),
- An activity indicator showing whether gesture detection is active or inactive,
- The currently detected gesture label,
- A confidence value associated with the detected gesture.

Even when gestures are inactive, the panel remains visible to clearly communicate system status rather than hiding functionality. This design choice improves transparency and user trust.

8.4.4 VOICE RECOGNITION PANEL

The Voice Recognition panel provides feedback from the voice pipeline.

Displayed information includes:

- Microphone listening status,
- A live audio level visualizer,
- The last recognized voice command,
- The inferred intent,
- Confidence values produced by the NLU stage.

This panel allows users to understand not only *what* the system heard, but *how* it interpreted the command, which is especially valuable during evaluation and debugging [9].

8.5 LIVE ACTIVITY LOG AND HISTORY VIEW

8.5.1 LIVE ACTIVITY LOG

The Live Activity Log continuously records events generated by both the UI and backend engines, including:

- System start and stop events,
- Wake word detection,
- Voice transcription output,
- Gesture recognition events,
- Executed system actions,
- Errors and warnings.

Logs are categorized into types such as VOICE, GESTURE, ACTION, and SYSTEM, allowing users to quickly understand the nature of each event.

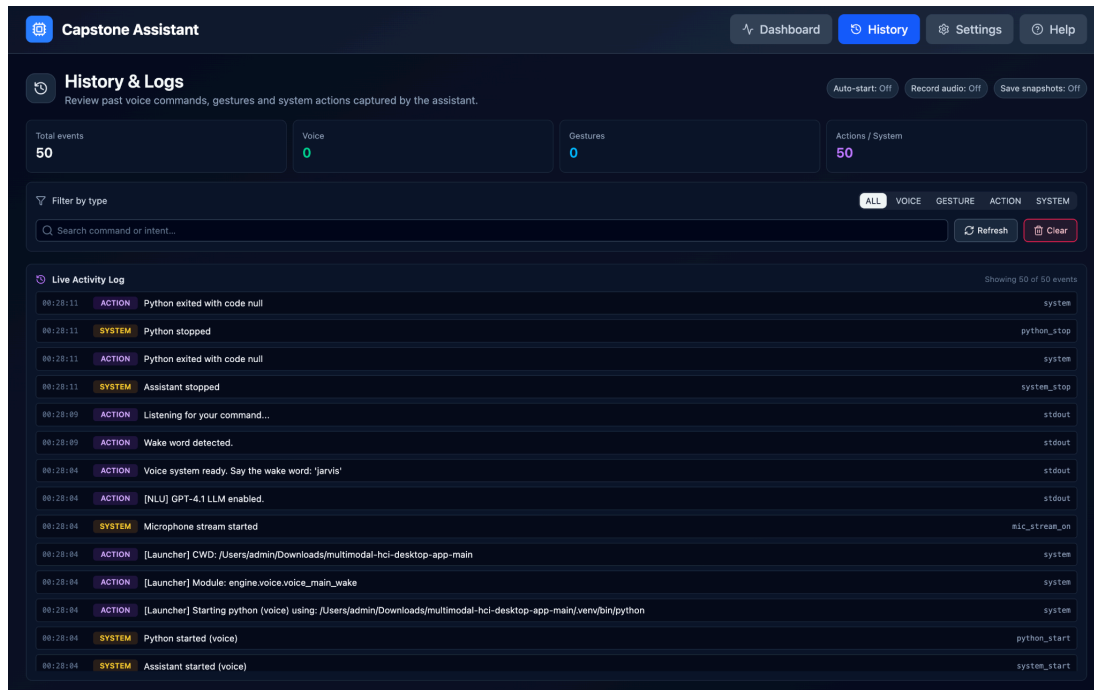


Fig14. Log and history UI

This panel allows users to understand not only *what* the system heard, but *how* it interpreted the command, which is especially valuable during evaluation and debugging.

8.5.2 HISTORY PAGE

The History page provides a persistent, searchable view of past system activity. It aggregates logs across sessions and presents summary statistics such as:

- Total number of events,
- Number of voice interactions,
- Number of gesture interactions,
- Number of system actions executed.

Filtering and search functionality allow users and evaluators to isolate specific interaction types, making this view particularly useful for user studies and post-session analysis.

8.6 SETTINGS PAGE AND SYSTEM CONFIRMATION

The Settings page allows users to configure the assistant without modifying code. Key configuration options include:

- Selecting microphone and webcam devices,
- Enabling or disabling gesture detection,
- Enabling auto-start behavior,
- Controlling logging behavior (audio recording, camera snapshots),
- Clearing stored logs.

This separation of configuration from core logic improves usability and ensures the system can adapt to different hardware environments and experimental setups.

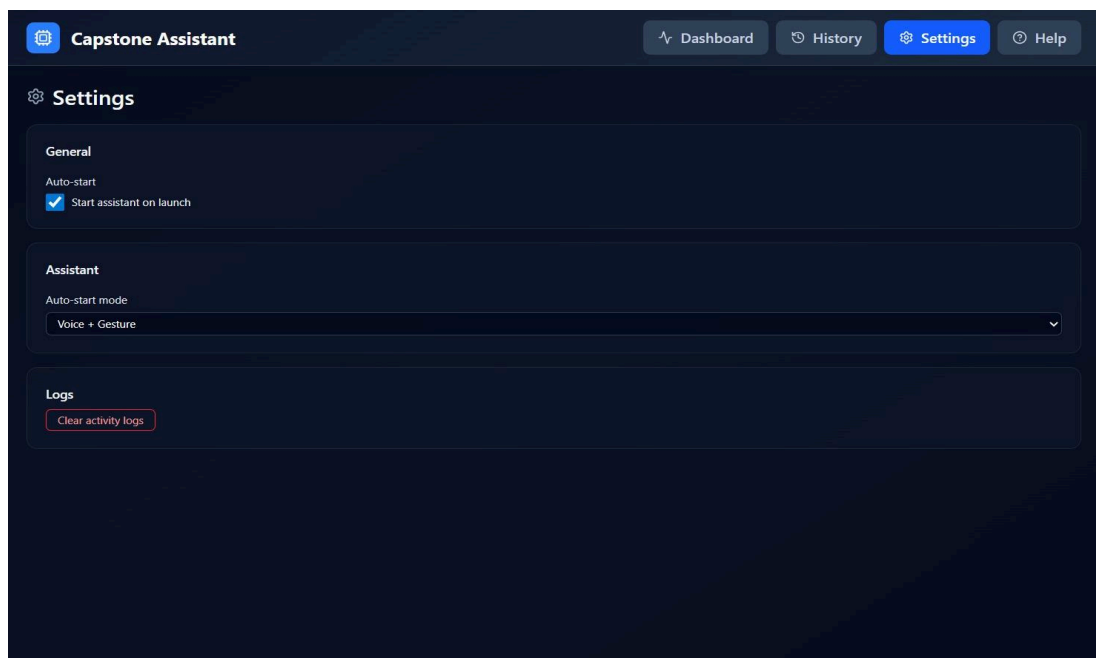


Fig15. Setting interface.

8.7 TUTORIAL AND USER GUIDANCE INTERFACE

To support usability and reduce onboarding time, the application includes a built-in Tutorial page.

The tutorial explains:

- What the assistant can do,
- How to issue voice commands naturally,
- How to correctly perform supported gestures,
- The mapping between gestures and system actions,
- Example voice commands and their interpreted intents.

This page ensures that new users can quickly understand system capabilities without external documentation and supports consistent interaction during demonstrations or evaluations.

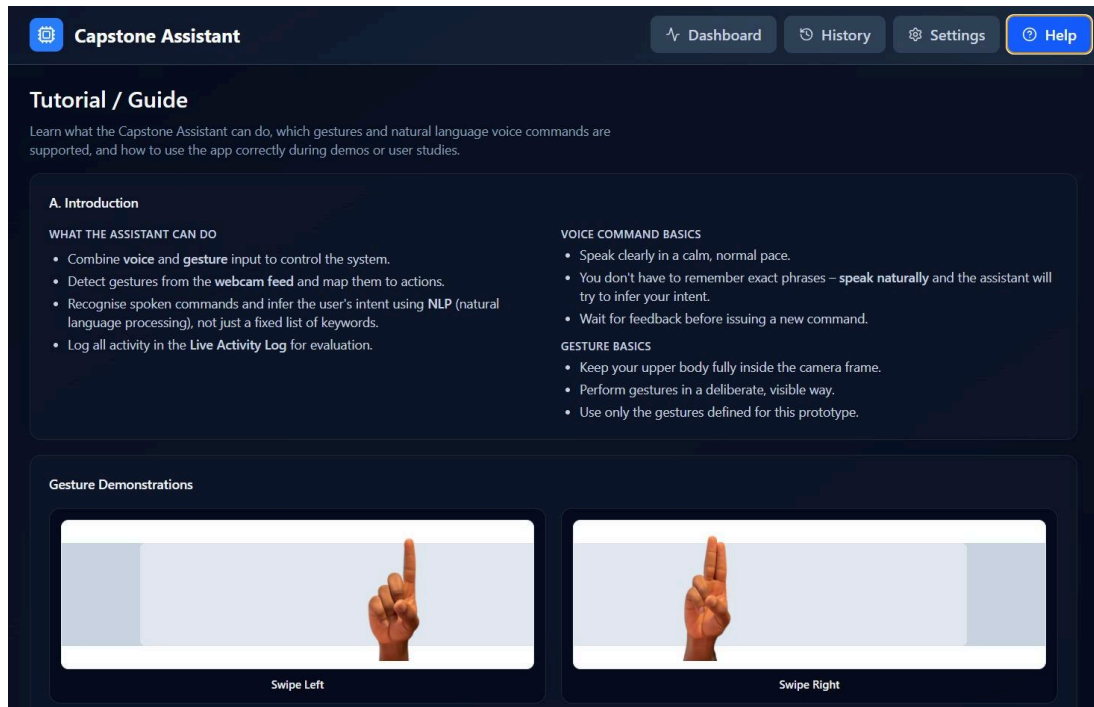


Fig16. Tutorial page

8.8 USER INTERACTION FLOW SUMMARY

From a user perspective, interaction with the system follows a clear and consistent flow:

1. Launch the application.
2. Select an operating mode (voice, gesture, or both).
3. Observe real-time feedback during interaction.
4. Review executed actions and recognition results in the log.
5. Adjust settings or consult the tutorial as needed.
6. Stop the assistant safely when finished.

This flow was designed to minimize ambiguity while maintaining flexibility for experimentation.

8.9 CHAPTER SUMMARY

This chapter detailed the design and implementation of the application interface used to control and visualize the multimodal interaction system. By combining a modern desktop UI with robust backend integration, the system provides not only powerful interaction capabilities but also transparency, configurability, and usability.

The interface plays a critical role in bridging low-level recognition algorithms with real-world user interaction, ensuring that the multimodal system is practical, observable, and suitable for evaluation and deployment.

CHAPTER NINE

SYSTEM INTEGRATION, TESTING, AND PERFORMANCE EVALUATION

9.1 FRONT-END AND BACK-END INTEGRATION

The connection between the front-end application and the back-end processing system is a core component of the multimodal interaction framework. The front end, implemented as a desktop application using Electron with a React-based user interface, serves as the primary point of interaction for the user. It provides controls for starting and stopping the system, selecting interaction modes (voice only, gesture only, or both), and visualizing system feedback in real time.

The back end consists of Python-based modules responsible for voice recognition, gesture recognition, intent interpretation, and action execution. Communication between the front end and back end is handled through inter-process communication (IPC), where the Electron main process launches and manages the Python runtime as a child process. Standard output and error streams from the Python process are captured and forwarded to the front end, allowing system events, recognition results, and errors to be displayed in the live activity log.

This architecture ensures a clean separation of concerns: the front end focuses on user experience and visualization, while the back end handles computationally intensive tasks such as audio processing, computer vision, and natural language understanding. The modular integration also allows individual components to be modified or extended without affecting the overall system structure.

9.2 TESTING METHODOLOGY

Testing was conducted incrementally throughout development to ensure stability and correctness across all system components. Individual modules were first tested in isolation, including wake word detection, speech transcription, intent prediction, gesture classification, and action dispatching. Once individual components were verified, integration testing was performed to validate communication between the front end and back end.

User-level testing involved running the system in each operational mode voice-only, gesture-only, and combined multimodal mode—to confirm that the correct modules were activated and deactivated as expected. The live activity log was used as a diagnostic tool to verify system state transitions, recognized inputs, and executed actions. This approach allowed rapid identification of errors and inconsistencies during runtime.

9.3 DEBUGGING AND SYSTEM REFINEMENT

Debugging focused primarily on synchronization issues, resource management, and runtime stability. One major challenge involved ensuring that Python processes were properly terminated when the user stopped the system or switched interaction modes. This was resolved through explicit process lifecycle management within the Electron main process, ensuring that no orphaned processes remained active.

Additional debugging efforts addressed camera and microphone resource conflicts, particularly when switching between modalities. Careful handling of device streams and explicit cleanup procedures ensured that hardware resources were released correctly, preventing crashes or unresponsive behavior. Logging played a critical role during debugging, as detailed system messages allowed precise tracing of execution flow and error conditions.

9.4 PERFORMANCE EVALUATION

System performance was evaluated based on responsiveness, reliability, and real-time behavior. For voice interaction, latency was measured from wake word detection to command execution, with results demonstrating near real-time responsiveness suitable for interactive use. Gesture recognition performance was assessed by observing detection stability and classification consistency during continuous hand movement.

The system maintained acceptable performance on consumer-grade hardware, with real-time processing achieved without noticeable delays. Lightweight design choices, such as selective activation of modalities and modular execution, contributed to efficient resource usage and stable operation.

9.5 ACCURACY IMPROVEMENTS

Accuracy improvements were achieved through iterative refinement of both voice and gesture pipelines. For voice recognition, improvements focused on reducing false activations and misinterpretations by refining wake word detection parameters and enforcing confidence thresholds in intent prediction. The use of natural language understanding allowed flexible phrasing while maintaining reliable intent mapping.

Gesture accuracy was improved through careful gesture definition, controlled data collection, and real-time validation. Gestures that produced inconsistent results were either refined or removed, ensuring that only stable and easily distinguishable gestures were included in the final system. Confidence-based filtering further reduced unintended activations during continuous operation.

CHAPTER TEN

CHALLENGES, DESIGN EVOLUTION, AND SYSTEM LIMITS

10.1 DEVELOPMENT CHALLENGES

The development of a real-time multimodal human–computer interaction system introduced several technical and design challenges. One of the primary difficulties was achieving reliable performance across both voice and gesture modalities while maintaining low latency and system stability. Each modality introduced its own constraints, including microphone sensitivity, background noise, camera lighting conditions, and gesture variability across users.

Another significant challenge was managing system complexity as additional features were introduced. Integrating voice recognition, natural language understanding, gesture recognition, and operating system control into a single cohesive system required careful coordination between independent modules. Ensuring that these components could operate simultaneously without resource conflicts or unintended behavior was a recurring challenge throughout development.

10.2 EVOLUTION OF DESIGN DECISION

Several core design decisions evolved significantly over the course of the project as a result of testing, experimentation, and performance evaluation.

Initially, voice command interpretation relied on rule-based and fuzzy string matching techniques. While this approach was lightweight, it proved insufficient for handling natural language variability and complex user commands. Commands that deviated slightly from predefined patterns often resulted in misclassification or failure. As a result, this approach was abandoned in favor of a large language model–based natural language understanding pipeline, which provided far greater flexibility and robustness.

The speech-to-text component also underwent multiple revisions. Early experiments included the use of Whisper for transcription; however, practical testing revealed limitations in responsiveness and consistency for real-time command execution within the project's constraints. To address this, the system transitioned to AssemblyAI, which offered more reliable transcription accuracy and predictable latency for short spoken commands.

Another major design evolution involved the voice interaction paradigm itself. The initial implementation required users to press and hold a key while speaking commands. Although functional, this approach was unintuitive and detracted from the goal of seamless, hands-free interaction. This led to the introduction of a wake-word-based system, enabling fully automatic voice activation. The wake-word mechanism significantly improved usability and aligned the system more closely with real-world voice assistants.

Gesture recognition design also evolved over time. Early gesture sets included a larger number of gestures, some of which proved unreliable during continuous operation. Through iterative testing, the gesture vocabulary was refined to prioritize stability, distinctiveness, and ease of execution. Gestures that caused ambiguity or frequent false positives were removed or redesigned.

10.3 TRADE-OFFS AND DESIGN CONSTRAINTS

Throughout development, several trade-offs were consciously made to balance performance, accuracy, and system complexity. One such trade-off involved limiting the number of supported gestures and voice intents to ensure reliable recognition. While a larger interaction vocabulary was desirable, prioritizing accuracy and responsiveness resulted in a more dependable system overall.

Another trade-off concerned dependency selection. The use of external APIs for speech transcription and language understanding improved accuracy but introduced reliance on network connectivity and third-party services. This decision was made to

achieve high-quality recognition within the scope of the project, with the understanding that future iterations could explore fully offline alternatives.

Performance optimization also required balancing real-time responsiveness against computational load. Lightweight configurations and selective module activation were employed to ensure smooth operation on consumer-grade hardware, even when both voice and gesture modalities were active simultaneously.

10.4 SYSTEM LIMITATIONS

Despite its functionality, the current system has several limitations. The reliance on external services for speech recognition and natural language understanding means that performance may degrade under poor network conditions or API availability constraints. Additionally, the system is currently optimized for macOS, limiting portability to other operating systems without modification.

Gesture recognition accuracy is influenced by environmental factors such as lighting conditions, camera placement, and background clutter. While the system performs reliably under controlled conditions, extreme variations can reduce recognition consistency. Furthermore, the system currently supports single-user interaction and does not account for multiple users within the camera frame.

10.5 FUTURE WORK

The limitations identified in this project present clear opportunities for future development. One potential direction is the integration of fully offline speech recognition and natural language understanding models to reduce dependency on external services. Expanding cross-platform support would also enhance system accessibility and applicability.

Future work could also explore adaptive gesture learning, allowing the system to personalize gesture recognition based on individual user behavior. Additional

multimodal fusion strategies, such as combining gesture confidence with voice context, could further improve interaction reliability. Finally, usability studies involving a broader range of users would provide valuable insights into real-world performance and inform further refinements.

10.6 SUMMARY

This chapter examined the challenges encountered during development, the evolution of key design decisions, and the limitations of the current system. Through iterative refinement and informed trade-offs, the project progressed from early prototypes to a robust multimodal interaction system. The lessons learned during development not only shaped the final implementation but also highlighted clear directions for future enhancement and research.

CONCLUSION

This project set out to design and implement a lightweight, real-time multimodal human–computer interaction system that enables users to control a computing device through a combination of voice commands and hand gestures. By integrating speech recognition, natural language understanding, computer vision–based gesture recognition, and operating system control, the system demonstrates how intuitive and natural interaction paradigms can replace or complement traditional input devices such as keyboards and mice.

The final system successfully achieves its core objectives. Voice interaction is handled through a fully automated wake-word–driven pipeline, eliminating the need for manual activation and allowing hands-free operation. Spoken commands are transcribed accurately and mapped to high-level intents using a language model–based natural language understanding module, enabling flexible interpretation of natural language rather than rigid command phrases. In parallel, the gesture recognition pipeline provides real-time cursor control and discrete gesture-based actions using a standard webcam and a carefully designed gesture vocabulary.

A key strength of this project lies in its modular architecture. Voice recognition, gesture recognition, system dispatch, and user interface components are decoupled, allowing each subsystem to operate independently while contributing to a unified interaction experience. This modularity not only improves maintainability and debugging but also allows future enhancements, such as additional modalities or alternative recognition engines, to be integrated with minimal redesign.

The development process emphasized iterative refinement and practical evaluation. Several design decisions evolved over time in response to observed limitations, including the transition from fuzzy matching to large language model–based intent recognition and from push-to-talk voice input to a fully automatic wake-word system. These changes significantly improved usability, accuracy, and overall user

experience, reinforcing the importance of experimentation and real-world testing in interactive system design.

While the system demonstrates strong performance under typical operating conditions, certain limitations remain. Dependence on external services for speech recognition and language understanding introduces network and availability constraints, and gesture recognition performance is influenced by environmental factors such as lighting and camera placement. Nevertheless, these limitations do not detract from the system's core contribution as a functional and extensible multimodal interaction framework.

In conclusion, this project demonstrates the feasibility and effectiveness of combining voice and gesture modalities for natural human–computer interaction. The resulting system provides a solid foundation for future research and development in multimodal interfaces, accessibility technologies, and intelligent user interaction systems. By bridging human communication patterns with computational control, the project contributes to the ongoing shift toward more intuitive, inclusive, and human-centered computing experiences.

REFERENCES

1. Oviatt, S. (2003). Multimodal Interfaces. In *The Human–Computer Interaction Handbook* (pp. 286–304). Lawrence Erlbaum Associates.
2. Turk, M. (2014). Multimodal interaction: A review. *Pattern Recognition Letters*, 36, 189–195. <https://doi.org/10.1016/j.patrec.2013.07.003>
3. Mitra, S., & Acharya, T. (2007). Gesture recognition: A survey. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 37(3), 311–324. <https://doi.org/10.1109/TSMCC.2007.893280>
4. Wachs, J. P., Kölsch, M., Stern, H., & Edan, Y. (2011). Vision-based hand-gesture applications. *Communications of the ACM*, 54(2), 60–71. <https://doi.org/10.1145/1897816.1897838>
5. Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed.). Pearson Education.
6. Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 1877–1901.
7. Lugaresi, C., Tang, J., Nash, H., et al. (2019). MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*. <https://arxiv.org/abs/1906.08172>
8. Picovoice Ltd. (2024). Porcupine Wake Word Engine Documentation. Available at: <https://picovoice.ai/docs/> (Accessed: 2025).
9. Cohen, M. H., Giangola, J. P., & Balogh, J. (2004). *Voice User Interface Design*. Addison-Wesley.
10. Norman, D. A. (2013). *The Design of Everyday Things* (Revised and Expanded Edition). Basic Books.

11. Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., Elmqvist, N., & Diakopoulos, N. (2016). *Designing the User Interface: Strategies for Effective Human–Computer Interaction* (6th ed.). Pearson.
12. Cheng, T. (2018). *Electron in Action: Building Cross-Platform Desktop Apps*. Manning Publications.
13. Google Research. (2024). MediaPipe Hands: Real-Time Hand Tracking. Available at: https://developers.google.com/mediapipe/solutions/vision/hand_landmarker (Accessed: 2025).
14. Yankelovich, N. (1996). How do users know what to say? *Communications of the ACM*, 39(4), 35–40. <https://doi.org/10.1145/227210.227217>

APPENDIX A

SAMPLE FIGURES



Fig17. Move cursor gesture

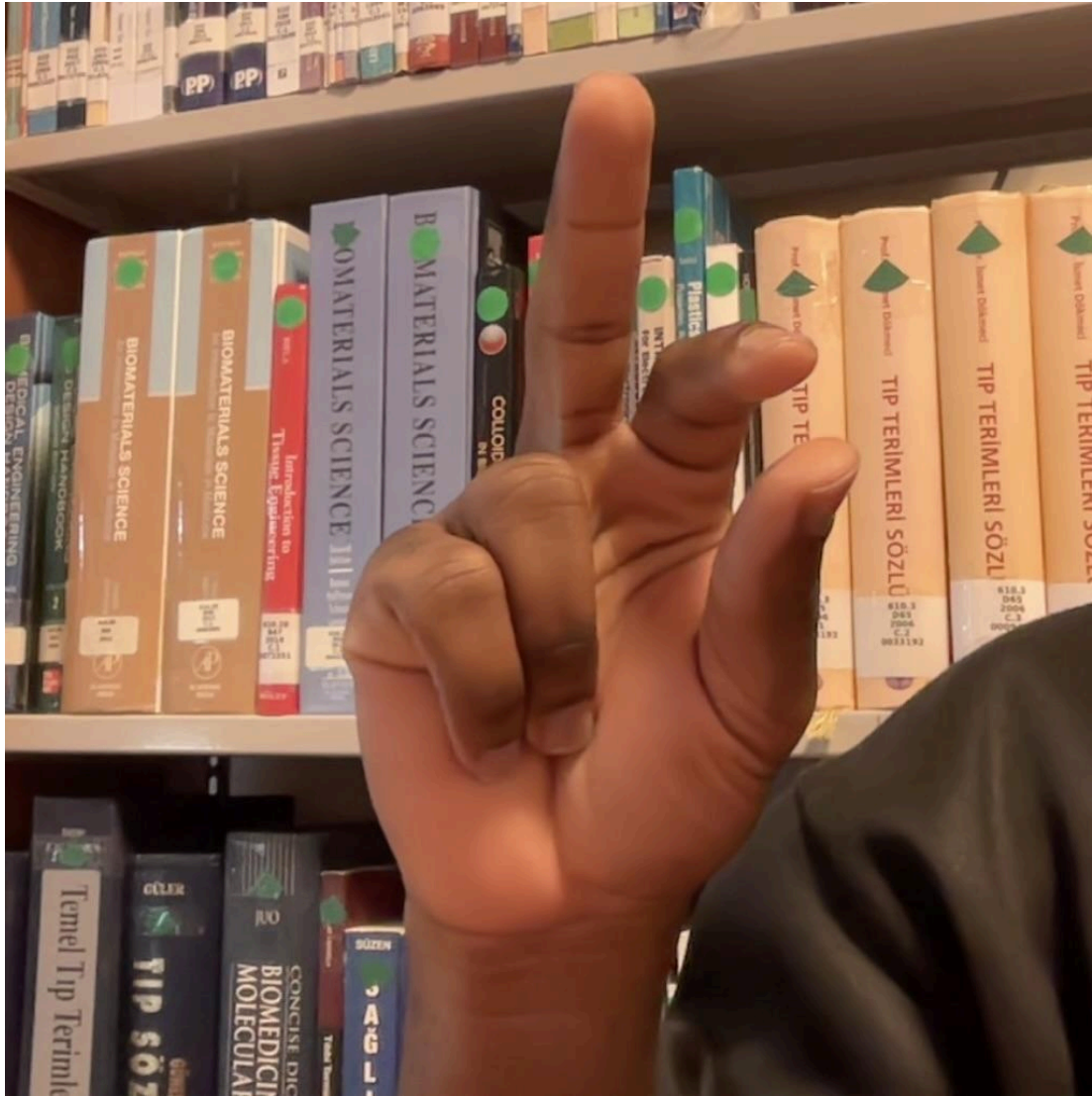


Fig18. Left click gesture.



Fig18. Right click gesture.



Fig19. Screenshot gesture



Fig21. Continuous scrolling up gesture

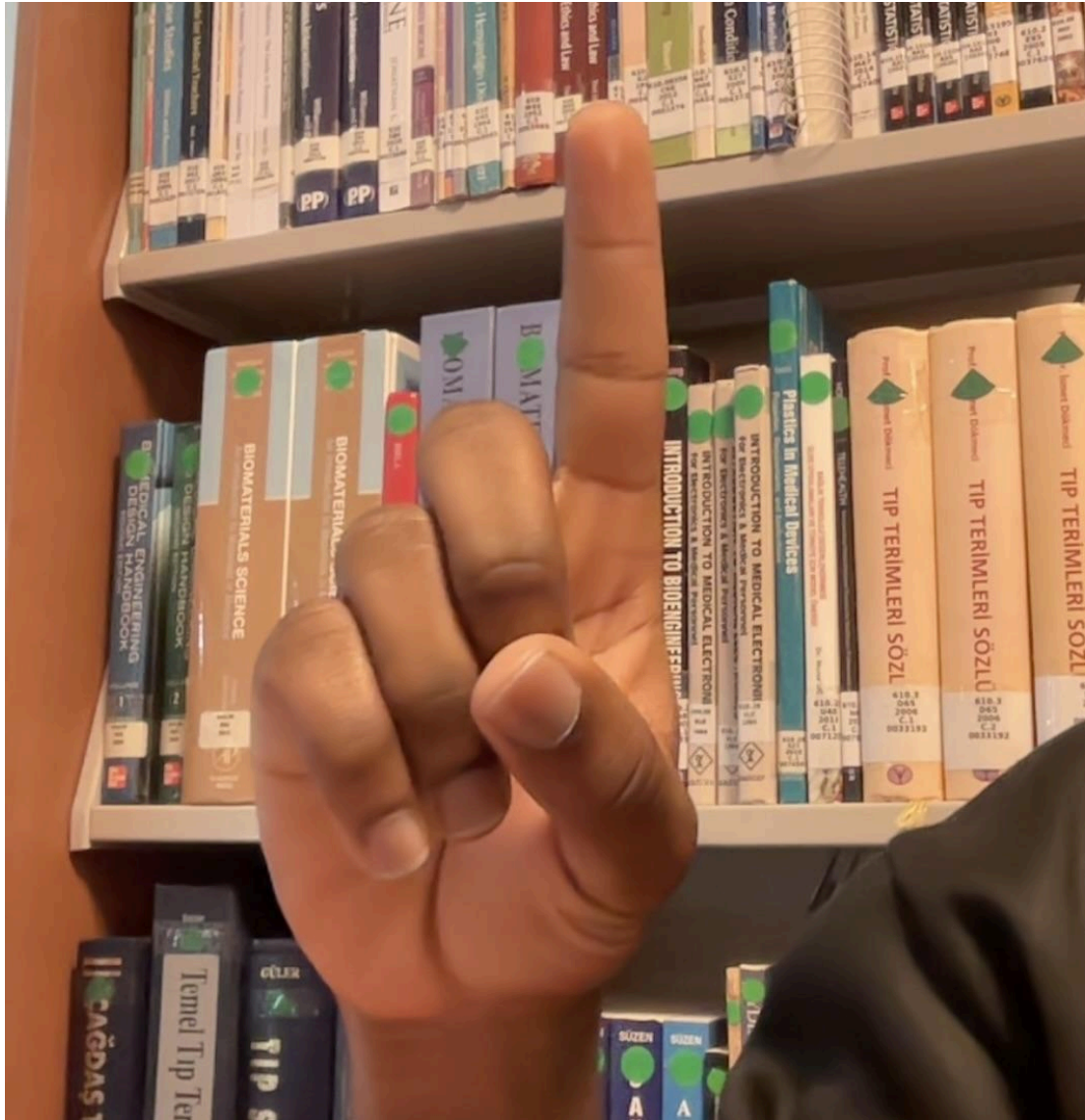


Fig22. Swipe left motion gesture



Fig23. Swipe Right motion gesture